Does bijectivity of g decide the Composite gain?

A short report on one experiment

The question in four sentences

Our benchmarks show Composite beating Direct by very different amounts, and we want to know what separates the cases. The proposition says that if g is *bijective*, observing the intermediates h tells you nothing you did not already learn from the objective y. So a natural guess is: **Composite helps when g is not bijective, and not otherwise.** This note tests that guess, and the answer is **no — bijectivity is not what decides it**.

1 What we built

A family of outer maps that are all the same shape of thing — products of linear factors — so that we can turn one property at a time:

$$g_m(z) = \prod_{i=1}^{m} (z - a_i), \qquad m = 1, \ldots, 5,$$

with the roots a_i spread across the range that h actually visits. Against each of these we put a map of the *same polynomial degree* that is bijective: z^3 and z^5 are strictly increasing, so they are just as nonlinear and just as steep, but nothing is lost. Plus e^z and $\sinh z$ as two more bijective references.

That matched pairing is the point. A degree-5 polynomial is both more non-bijective *and* a wilder function; if we only varied m we could never say which mattered.

2 Measuring how non-bijective each one is

Rather than assume it from the degree, we count it. For each map, over the values of y the problem actually produces,

$$\bar{n} = \mathbb{E}\left|g^{-1}(y)\right| = \text{average number of } h \text{ values consistent with one observation.}$$

$\bar{n} = 1$ means bijective where it matters. The five bijective maps all come out at exactly 1.00; the others at 2.00, 2.82, 3.75 and 4.50. So the knob works, and it is measured rather than asserted.

3 Why we stopped measuring optimisation

We first compared the two models by how well they optimise. That turned out to be the wrong instrument, for a reason that cannot be fixed by choosing better roots:

> **A bijective g is monotone. A monotone function attains its minimum at an endpoint.**

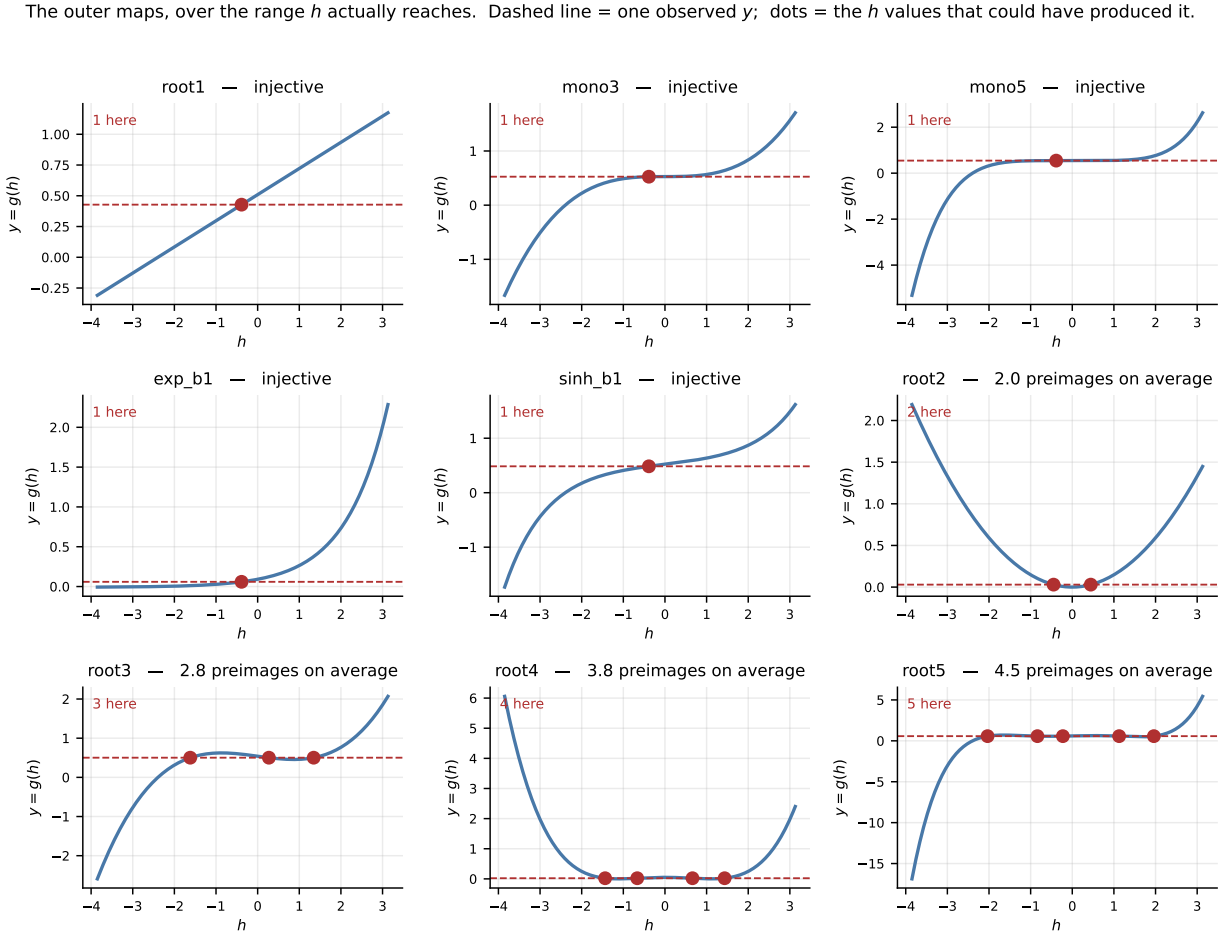


Figure 1: Every map in the family. The dashed line is one observed value of y; the dots are the values of h that could have produced it. One dot means y tells you h exactly — that is what bijective means. Several dots mean it does not.

So a bijective map always puts the optimum at the edge of the reachable range, and a non-bijective one can put it in the middle. Bijectivity decides *what the optimisation problem is*, which means any comparison of optimisation performance between the two classes is confounded before it starts.

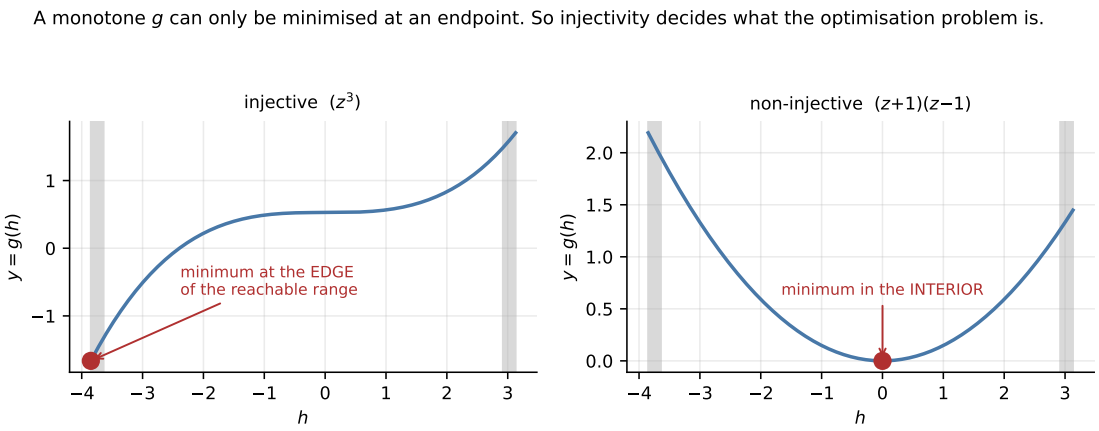


Figure 2: The same point, drawn. Grey bands mark the edges of the range h reaches.

4 What we measured instead

Prediction, with no optimiser anywhere. Both models are fitted to the same n observations and then asked to predict f at 512 held-out points. We score the whole predictive distribution with CRPS, a proper scoring rule, so a model is rewarded for honest uncertainty and not only for a good central guess. Thirty-two fits per map per sample size, at $n = 5, 10, 20, 40$.

5 The result

Two things stand out.

The matched pairs show no effect. This is the comparison the whole design was built for: same degree, same steepness, opposite bijectivity.

Degree	Bijective		Non-bijective		Difference	
3	`mono3`	$+0.0189 \pm 0.0022$	`root3`	$+0.0221 \pm 0.0040$	$+0.003 \pm 0.005$	(0.7 SE)
5	`mono5`	$+0.0367 \pm 0.0098$	`root5`	$+0.1133 \pm 0.0682$	$+0.077 \pm 0.069$	(1.1 SE)

Neither difference is distinguishable from zero.

The linear map is the real special case. `root1` has the smallest gain at every single sample size, four to twenty times below everything else.

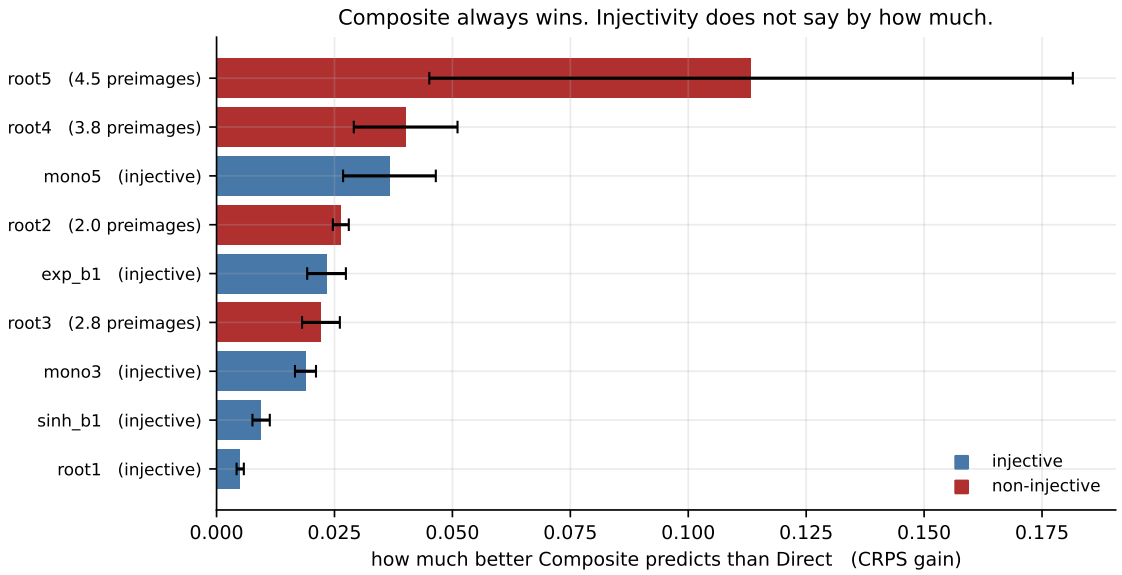


Figure 3: Composite predicts better than Direct everywhere. But the blue (bijective) and red (non-bijective) bars are completely interleaved — knowing whether g is bijective does not tell you how big the gain will be.

6 What this means

Bijectivity does control information — that part is proven. If g is bijective, observing h adds exactly nothing; we also confirmed it numerically to six decimal places. That is not in question.

But information is not what the gain is made of. Composite still beats Direct on bijective maps, by a margin several standard errors from zero. The reason is that Direct is fitting the *wrong kind of model*: when g is nonlinear, $f = g(h)$ is not a Gaussian process at all, so a GP placed directly on f is misspecified. Composite's model is correct by construction. Bijectivity removes the information advantage and leaves this one untouched.

So the gain has two sources, and we measured their sum.

Source of the gain	Removed by	Size here
Direct is fitting the wrong model	g being *linear*	the large part
Composite sees information y lacks	g being *bijective*	too small to detect

Linearity kills both, which is why `root1` is lowest, and why Truss and power flow — both linear — show the smallest gains in the paper. Bijectivity kills only the second, which is why the bijective maps still gain.

One sentence in the paper should be adjusted. "Even a nonlinear composition adds no information advantage when it is invertible" is correct about *information*, but it reads as a prediction that invertible maps will show no gain. They do show one. On this evidence, how nonlinear g is predicts the gain better than whether it is invertible.

7 What would settle it

Separating the two sources needs a third model: one that holds the correct prior on h but sees only y. Comparing that against Composite isolates the information channel with the misspecification held fixed, which is the only place bijectivity can show up cleanly. That is the three-arm setup, and this experiment is an argument for running it rather than a replacement.

Caveats

- One function family, $d = 6$, single objective, noiseless. The paper's hypervolume benchmarks are none of those things.
- `root5`'s error bar is enormous (± 0.068 on $+0.113$); a few outlier fits dominate it. The degree-3 pair is the trustworthy comparison.
- $\bar{n}$ counts branches without weighting them by how plausible they are, so a map with a technically reachable but very unlikely second branch looks more ambiguous than it behaves.